Теория графов

Плотников Даниил Михайлович

Санкт-Петербургский государственный университет

Оглавление

Определения 4
- Граф 5
- Полный граф 6
- Разряженные и плотные графы 7
- Пути 8
- Связность 9
- Дерево 10
- Лоза 11
- Ориентированный граф 12
- Взвешенный граф 13
- Смежность 14
- Смежность в орграфах 15
- Двудольность 16

Хранение графа 17
- Список смежности 18
- Матрица смежности 19
- Матрица инцидентности . . . 20
- Сравнение 21
- Когда что использовать? . . . 22

Поиск в глубину 24
- Определение 25
- Рекурсивная реализация . . . 27
- Итеративная реализация . . . 28

Сложность 30
Компоненты связности 32
Топологическая сортировка 36
Поиск циклов 41
Проверка двудольности графа 46
Поиск в ширину 50
Определение 51
Реализация 53
Сложность 54
Компоненты связности 56
Лабиринт 60
Кратчайший путь в невзвешенном графе 67

Определения

Граф

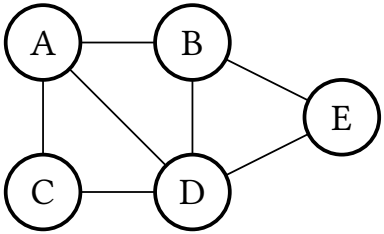
Обыкновенный граф – совокупность множеств $G(V, E)$, где

- V (Vertex) – множество вершин
- E (Edge) – Множество ребер
- $E \subseteq V^2 = \{\{u, v\} \mid u, v \in V; u \neq v\}$

Например:

$V = \{\mathrm{A}, \mathrm{B}, \mathrm{C}, \mathrm{D}, \mathrm{E}\}$,

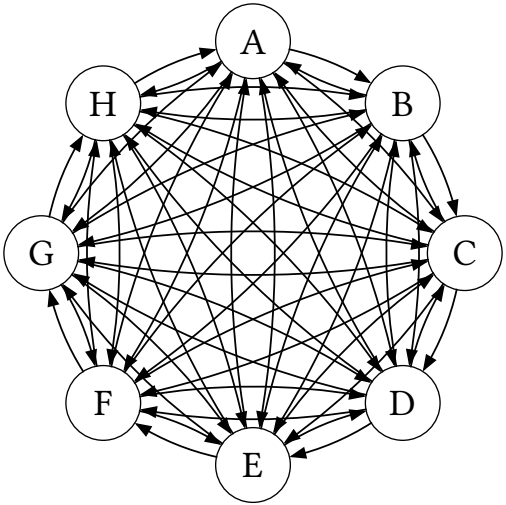
$E = \{(\mathrm{A}, \mathrm{B}), (\mathrm{A}, \mathrm{C}), (\mathrm{B}, \mathrm{E}), (\mathrm{C}, \mathrm{D}), (\mathrm{D}, \mathrm{A}), (\mathrm{D}, \mathrm{B}), (\mathrm{E}, \mathrm{D})\}$



Полный граф

Полный граф K_n — граф в котором каждая пара вершин соединена ребром.

Полный граф K_n имеет C_n^2 ребер. Каждая вершина имеет степень $n-1$.

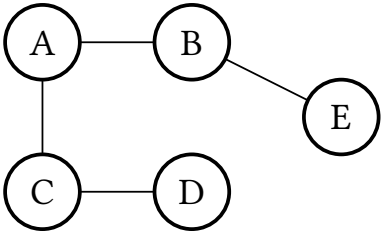


Разряженные и плотные графы

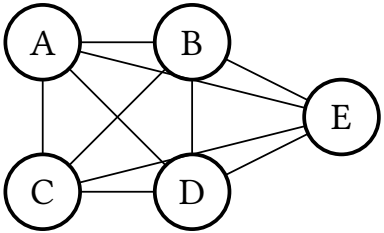
Разряженный граф - граф с относительно небольшим числом рёбер сравнительно с числом вершин (обычно $O(V)$).

Плотный граф - граф с большим числом рёбер, близким к полному (обычно $O(V^2)$).

Разряженный:



Плотный:



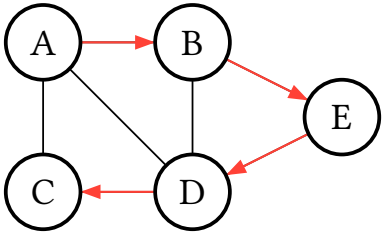
Пути

Путь ведет из одной вершины в другую и проходит по ребрам графа.

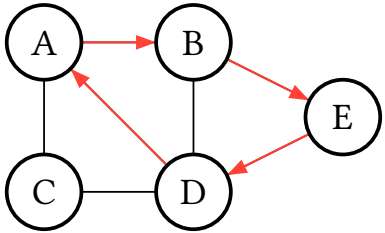
Длиной пути называется количество ребер в нем.

Циклом называется путь, в котором последняя вершина совпадает с первой.

Путь:

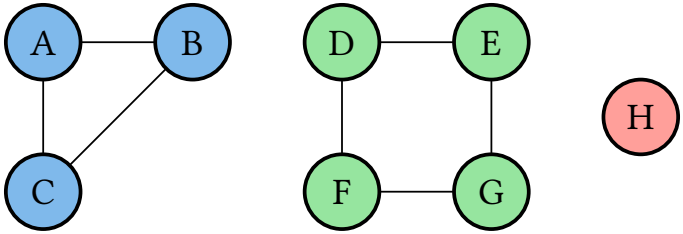


Цикл:



Связность

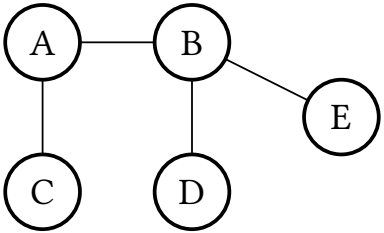
Граф называется **связным**, если между любыми двумя вершинами существует путь. Связные части графа называются его **компонентами связности**.



Компонены связности на графе сверху: $\{A, B, C\}$, $\{D, E, F, G\}$, $\{H\}$

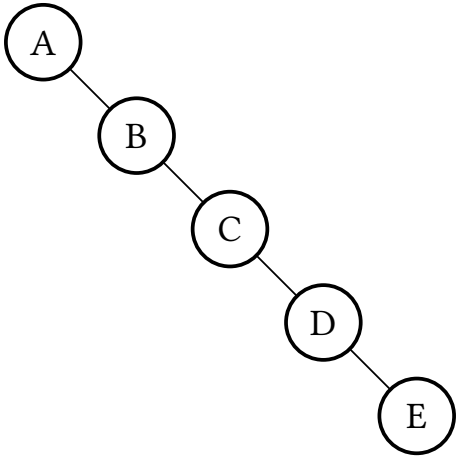
Дерево

Дерево - связный граф без циклов.



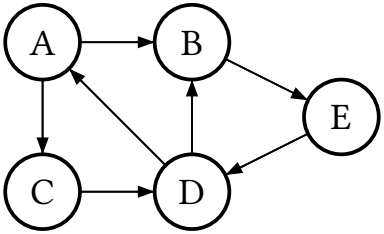
Лоза

Лоза (бамбук) - дерево, в котором каждая вершина имеет степень не более двух, представляющее собой линейную структуру (цепь вершин без ветвления).



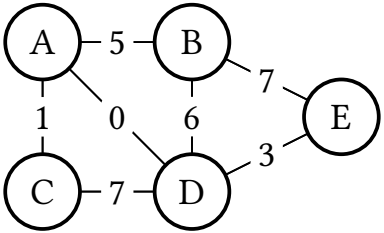
Ориентированный граф

В **ориентированном графе** по каждому ребру можно проходить только в одном направлении.



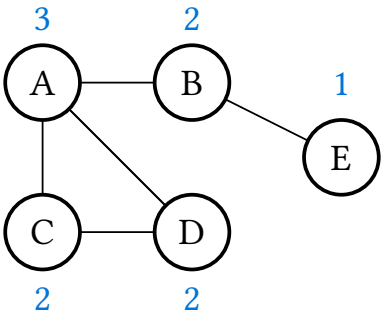
Взвешенный граф

Во **взвешенном графе** каждому ребру сопоставлен вес. Часто веса интерпретируются как длины ребер, а длиной пути считается сумма весов составляющих его ребер.



Смежность

Две вершины называются **соседними**, или **смежными**, если они соединены ребром. **Степенью** вершины называется число соседних с ней вершин



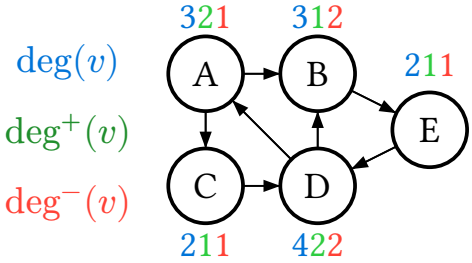
Смежность в орграфах

В ориентированных графах степень вершины делится на **полустепень захода** (in-degree) и **полустепень выхода** (out-degree).

Полустепень захода $\deg^-(v)$ — число входящих ребер в вершину v.

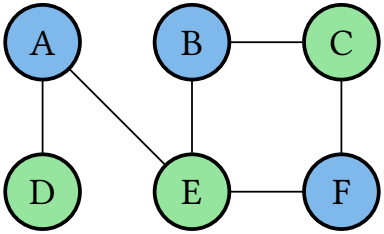
Полустепень выхода $\deg^+(v)$ — число исходящих ребер из вершины v.

Общая степень: $\deg(v) = \deg^-(v) + \deg^+(v)$.



Двудольность

Граф называется **двудольным**, если его вершины можно раскрасить в два цвета, так что цвета любых двух соседних вершин различны.

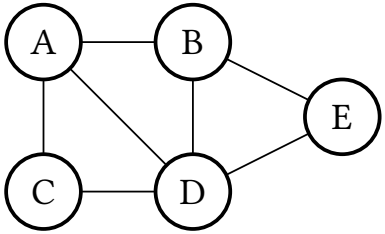


Хранение графа

Список смежности

Список смежности — один из способов представления графа в виде коллекции списков вершин. Каждой вершине графа соответствует список, состоящий из «соседей» этой вершины.

A	B, C, D
B	A, D, E
C	A, D
D	A, B, C, E
E	B, D



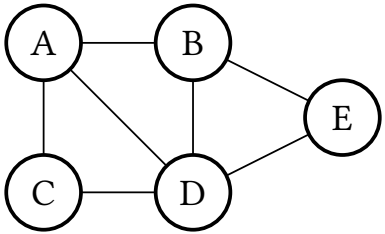
В ориентированном случае обычно хранят ребро только для вершины из которой оно исходит. Во взвешанном случае так же в пару к вершине записывают вес.

Матрица смежности

Матрица смежности — один из способов представления графа в виде матрицы.

$$A_{i,j} = \begin{cases} 1 \text{ if } (i,j) \in E \\ 0 \end{cases}$$

	A	B	C	D	E
A	0	1	1	1	0
B	1	0	0	1	1
C	1	0	0	1	1
D	1	1	1	0	1
E	0	0	1	1	0

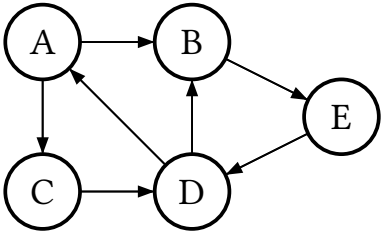


В ориентированном случае обычно хранят ребро только для вершины из которой оно исходит. Во взвешанном случае вместо единицы записывают вес.

Матрица инцидентности

Матрица инцидентности — матрица размера $V \times E$, где строки соответствуют вершинам, столбцы — ребрам. Элемент матрицы равен 1, если вершина инцидентна ребру (для ориентированных графов: 1 для исходящего ребра, −1 для входящего).

	(A,B)	(A,C)	(B,E)	(C,D)	(D,A)	(D,B)	(E,D)
A	1	1	0	0	−1	0	0
B	−1	0	1	0	0	−1	0
C	0	−1	0	1	0	0	0
D	0	0	0	−1	1	1	−1
E	0	0	−1	0	0	0	1



В неориентированном случае обычно хранят 1 вместо −1. Во взвешанном случае вместо единицы записывают вес.

Сравнение

Операция	Матрица смежности	Список смежности	Матрица инцидентности
Память	$O(V^2)$	$O(V+E)$	$O(V \times E)$
Проверка существования ребра	$O(1)$	$O(V)$	$O(E)$
Получение списка соседей	$O(V)$	$O(1)$	$O(E)$
Добавление ребра	$O(1)$	$O(1)$[1]	$O(V \times E)$
Удаление ребра	$O(1)$	$O(V)$	$O(V \times E)$
Добавление вершины	$O(V^2)$	$O(1)$	$O(E)$
Удаление вершины	$O(V^2)$	$O(E)$	$O(V \times E)$

[1]* Амортизированная временная сложность для списка смежности при добавлении ребра без проверки дубликатов.

Когда что использовать?

Список смежности:

- Для разреженных графов (малое число рёбер).
- Когда требуется быстрый обход соседей вершин.
- Подходит для динамических графов (частое добавление/удаление рёбер).
- Рекомендуется как дефолт для большинства задач на графы.

Матрица смежности:

- Для плотных графов (большое число рёбер близкое к V^2).
- Когда нужны быстрые проверки существования рёбер между любыми двумя вершинами.
- Полезна если граф редко изменяется, но расчёты проводятся многократно.
- Хороша для небольших графов из-за квадратичного потребления памяти.

Матрица инцидентности:

- Для теоретических расчётов (ранг графа, линейные теоремы).
- Когда анализируется инцидентность рёбер и вершин.
- В приложениях сетевого моделирования или физики.
- Редко используется на практике из-за потребления памяти и сложности изменений.

Общие рекомендации:

- Выбирать представление исходя из типичных операций (обход, проверка рёбер, изменения).
- Учитывать ограничения по памяти — список смежности наиболее экономный.

В рамках соревновательного программирования, **список смежности** — почти всегда верный выбор

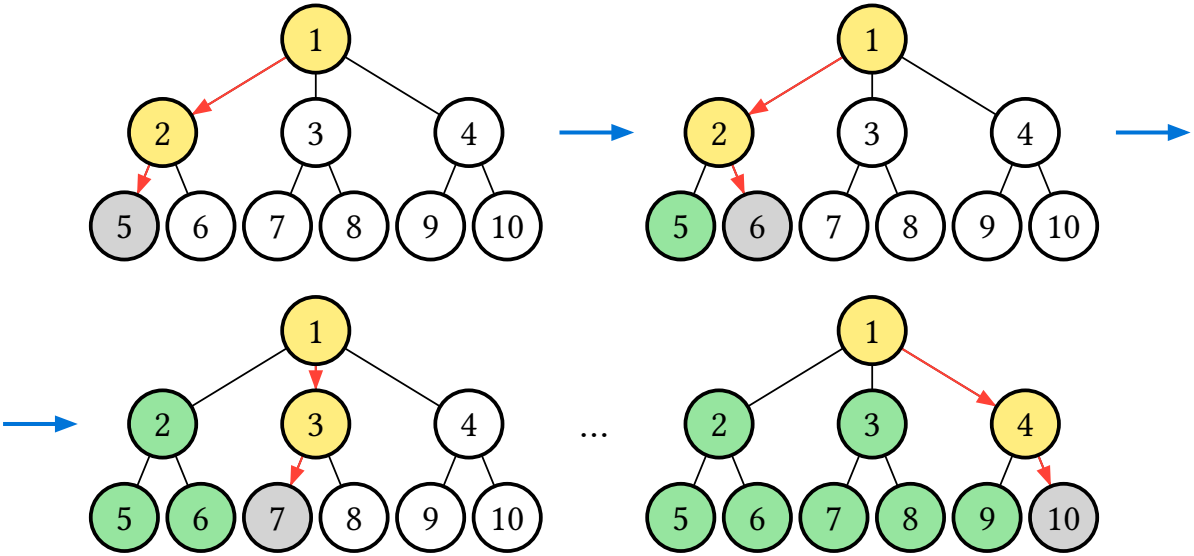
Поиск в глубину

Определение

Поиск в глубину (Depth-First Search, DFS) — метод обхода графа, при котором мы идем так глубоко, как возможно по ветке графа, прежде чем вернуться назад.

Алгоритм DFS начинает обход с некоторой выбранной вершины. Он посещает вершину, затем рекурсивно обходит всех её непосещённых соседей. Когда все соседи текущей вершины посещены, происходит возврат (backtracking) к предыдущей вершине.

Визуально DFS можно представить как погружение в глубину графа, аналогично лабиринту, где мы идем вглубь до тех пор, пока не достигнем тупика, а затем возвращаемся и ищем другие пути.



Порядок выхода из функции: $\{5, 6, 2, 7, 8, 3, 9, 10, 4, 1\}$

Такой порядок обхода называется **прямым (префиксным) обходом дерева**

Рекурсивная реализация

Рекурсивная реализация DFS использует стек вызовов функций для хранения текущего пути.

```
vector<vector<int>> adj; // список смежности
vector<char> visited;

void dfs(int v) {
    visited[v] = true;
    for (int u : adj[v]) {
        if (!visited[u]) {
            dfs(u);
        }
    }
}
```

Итеративная реализация

```
void dfs(int start){
    vector<char> visited(n, false);
    stack<int> s;
    s.push(start);
    visited[start] = true;

    while (!s.empty()){
        int v = s.top();
        s.pop();
        for (int u : adj[v])
            if (!visited[u]) {
                visited[u] = true;
                s.push(u);
            }
        }
    }
}
```

Если вы всё же используете python для спортивного программирования, то вы не можете позволить себе использовать рекурсивый dfs. Не будет хватать глубины рекурсии и затраты на вызов функции в питоне выше.

```
def dfs(start, adj, n):
    visited = [False] * n
    stack = [start]
    visited[start] = True

    while stack:
        v = stack.pop()

        for u in adj[v]:
            if not visited[u]:
                visited[u] = True
                stack.append(u)
```

Сложность

- **Временная сложность**: $O(V + E)$, где V — количество вершин, E — количество рёбер. Каждая вершина и каждое ребро посещается постоянное число раз.
- **Пространственная сложность**: $O(V)$ для хранения массива visited и стека/очереди рекурсии. В худшем случае (линейный граф) может потребоваться $O(V)$ стека.

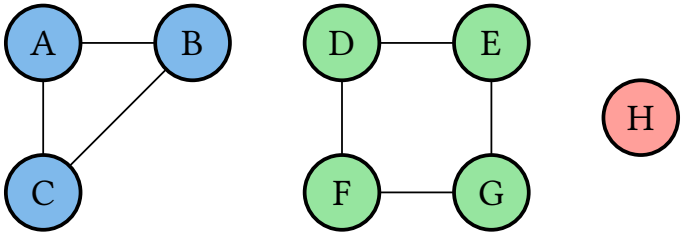
DFS работает оптимально по времени для большинства задач на графах.

Практические задачи

- **Parity**: Timus 1003
- **Путешествие**: Codeforces 1020B
- **И коровы приходят домой**: Codeforces 176F

Компоненты связности

Дан неориентированный граф. Требуется найти все компоненты связности, А так же определить к какой из компонент относится каждая вершина.



Компоненты связности графа сверху: $\{A, B, C\}$, $\{D, E, F, G\}$, $\{H\}$.

Для поиска компонент связности используем DFS: запускаем обход от каждой непосещённой вершины, каждый такой обход посещает одну компоненту связности.

Нам требуется пометить к какой из компонент относится вершина. Как можно уникально обозначить компоненту? Можно просто в качестве “названия” компоненты использовать номер любой из вершин лежашей в ней.

```
void dfs(int v, int u) {
    visited[v] = u;
    for (int i : adj[v]) {
        if (!visited[i]) {
            dfs(i, u);
        }
    }
}
```

```
vector<vector<int>> adj;
vector<int> visited;

int main(){
  //Считывание

  int n = adj.size();
  visited.assign(n, 0);
  for (int i = 0; i < n; ++i) {
    if (!visited[i]) {
      dfs(i, i);
    }
  }

  //Вывод
}
```

Работает за $O(V + E)$.

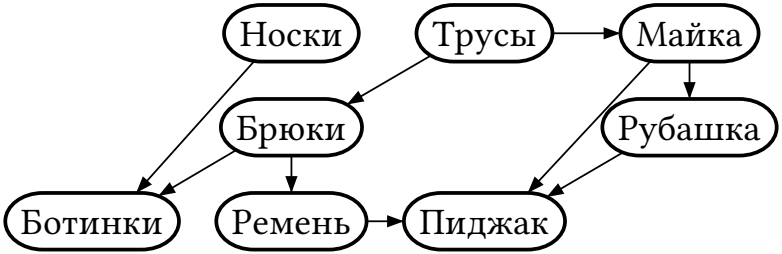
Практические задачи

- **Max Area of Island**: LeetCode 695
- **Компоненты графа**: acmp.ru 843
- **Поиск компонент связности**: Codeforces 920D
- **Присвоение компонент**: Codeforces 145E
- **Остров**: acmp.ru 47

Топологическая сортировка

Топологическая сортировка — упорядочение вершин ориентированного графа без циклов, при котором для каждого ребра (u, v) вершина u идёт перед v.

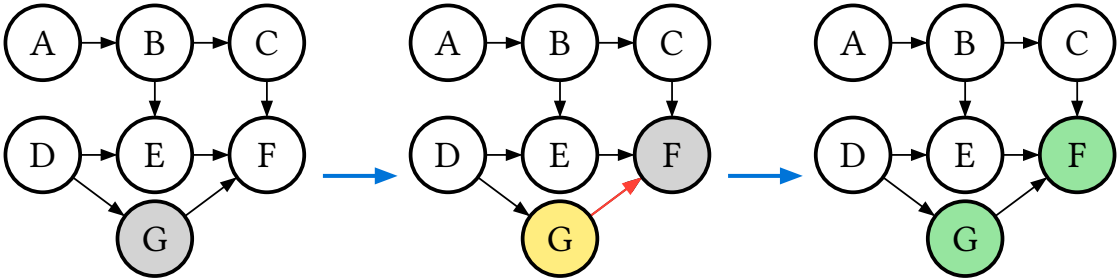
Дан ориентированный ацикличный граф (**DAG - Directed Acyclic Graph**). Требуется вывести топологическую сортировку вершин. Например, модель одевания одежды:



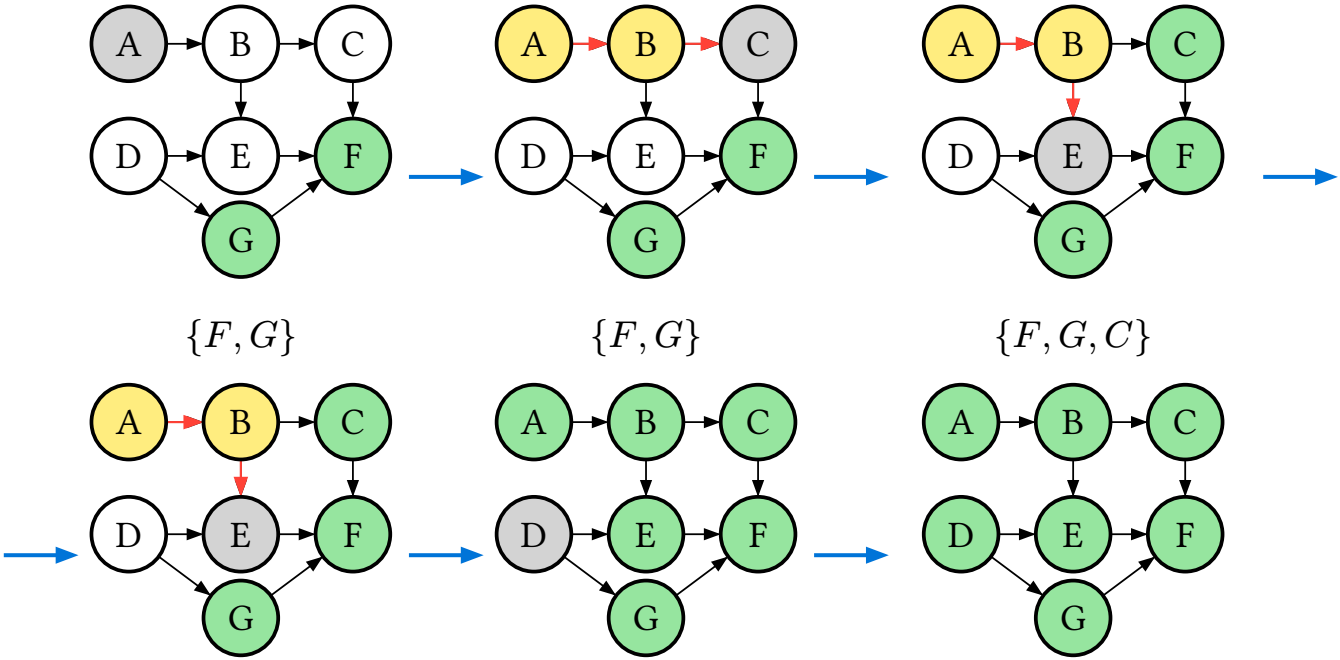
Возможный порядок: Носки, Трусы, Брюки, Майка, Ремень, Рубашка, Ботинки, Пиджак.

Решение

Для решения используем DFS: обходим граф, и добавляем вершину в список после посещения всех её соседей. Переворачиваем список для получения топологического порядка.



$\{\}$ $\{\}$ $\{F, G\}$


A B C D E F G
$\{F, G\}$
A B C D E F G
$\{F, G\}$
A B C D E F G
$\{F, G, C\}$
A B C D E F G
$\{F, G, C\}$
A B C D E F G
$\{F, G, C, E, B, A\}$
A B C D E F G
$\{F, G, C, E, B, A, D\}$

```
vector<vector<int>> adj;
vector<char> visited;
vector<int> order;

void dfs(int v) {
    visited[v] = true;
    for (int u : adj[v]) {
        if (!visited[u]) {
            dfs(u);
        }
    }
    order.push_back(v);
}
```

Работает за $O(V + E)$.

Практические задачи

- **Course Schedule**: LeetCode 207
- **Course Schedule II**: LeetCode 210
- **Parallel Courses**: LeetCode 1136
- **Topological Sort**: SPOJ “TOPOSORT”

Поиск циклов

Дан ориентированный граф. Требуется определить, содержит ли он цикл.

Цикл в графе — это путь, в котором первая и последняя вершина совпадают, и все рёбра уникальны.

В ориентированном графе наличие циклов означает, что некоторые алгоритмы не применимы.

Задача важна для проверки ацикличности (DAG — Directed Acyclic Graph), что является основой для многих задач, таких как планирование, зависимости в программном обеспечении и т.п.

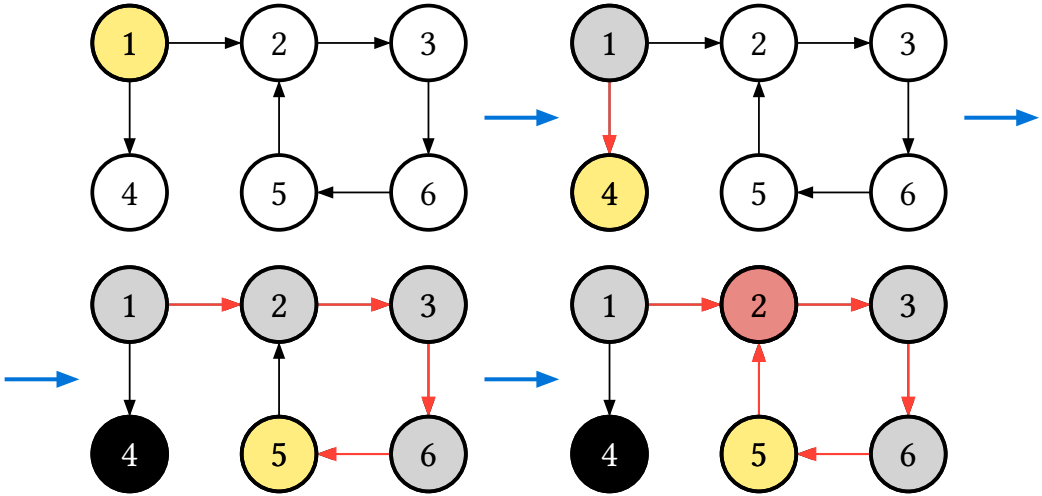
Для поиска циклов в ориентированном графе с помощью DFS мы используем три состояния вершин:

- **Белый** (не посещена)
- **Серый** (обрабатывается — находится в текущем пути обхода)
- **Чёрный** (посещена и обработана)

При обходе соседей из текущей вершины:

- Если сосед белый, запускаем DFS на нём
- Если сосед серый, обнаружен цикл
- Если сосед чёрный, продолжаем

Для неориентированных графов родитель должен быть исключён из проверки.


1
2
3
4
5
6
1
2
3
4
5
6
1
2
3
4
5
6
1
2
3
4
5
6

```
vector<vector<int>> adj;
vector<int> color; // 0 - white, 1 - grey, 2 - black

bool has_cycle(int v) {
    color[v] = 1; // grey
    for (int u : adj[v]) {
        if (color[u] == 0) {
            if (has_cycle(u)) return true;
        } else if (color[u] == 1) {
            return true; // back edge to grey
        }
    }
    color[v] = 2; // black
    return false;
}
```

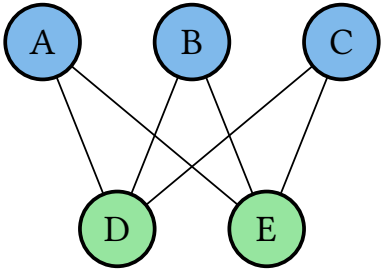
Работает за $O(V + E)$

Практические задачи

- **Find Eventual Safe States**: LeetCode 802
- **Redundant Connection**: LeetCode 684
- **Redundant Connection II**: LeetCode 685
- **Account Merge**: LeetCode 721

Проверка двудольности графа

Дан неориентированный граф. Требуется определить, является ли он двудольным.

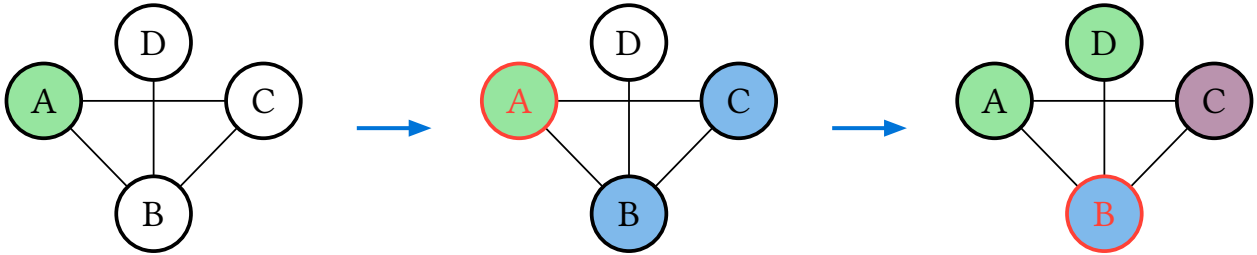


Граф выше является двудольным, вершины разделены на два множества: $\{A, B, C\}$ и $\{D, E\}$.

Эта проверка полезна во многих задачах соревновательного программирования, таких как задачи о назначении, моделирование противоречий или определение возможности разделения ресурсов на две группы без конфликтов.

Решения

1. Выбираем неокрашенную вершину, красим её в цвет 1
2. В процессе обхода красим соседей в противоположный цвет
3. Если обнаруживаем вершину, окрашенную в “неправильный” цвет — граф не является двудольным
4. Повторяем для всех компонент связности



```
vector<vector<int>> adj;
vector<int> color; // 0 - not colored, 1 - color1, 2 - color2

bool is_bipartite(int v, int c) {
    color[v] = c;
    for (int u : adj[v]) {
        if (color[u] == 0) {
            if (!is_bipartite(u, 3 - c)) return false;
        } else if (color[u] == c) {
            return false; // same color
        }
    }
    return true;
}
```

Работает за $O(V + E)$.

Практические задачи

- **Bipartite Graph Check**: acmp.ru 987
- **Двудольный граф**: acmp.ru 455
- **Graph Coloring**: SPOJ “COLOR”
- **Even Edge**: Timus 1837

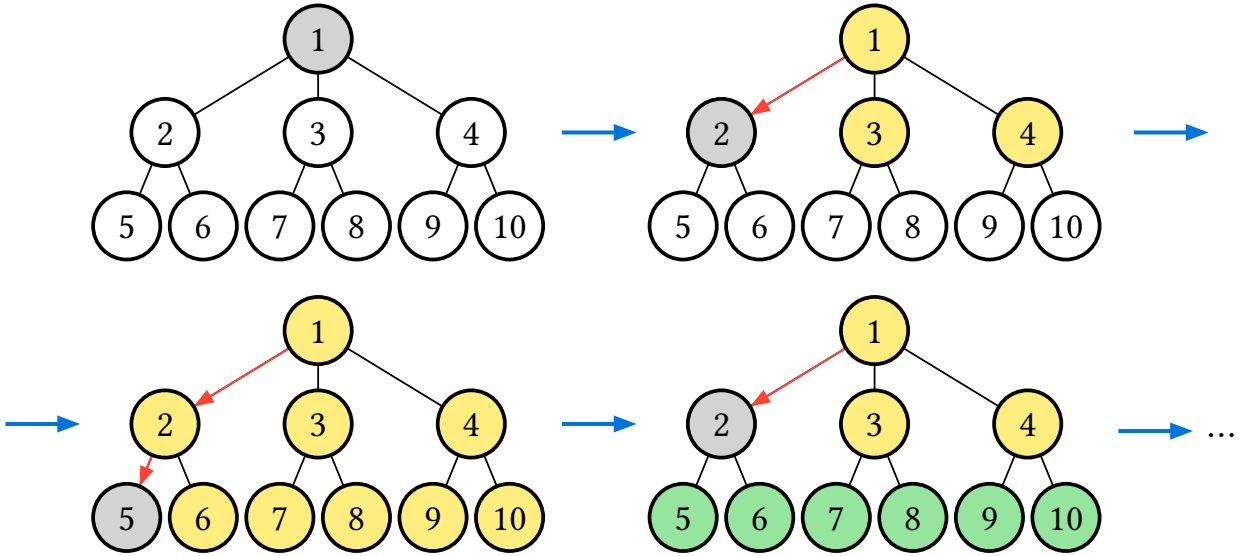
Поиск в ширину

Определение

Поиск в ширину (Breadth-First Search, BFS) — метод обхода графа, при котором мы идем слоями по ребрам графа, начиная с ближайших соседей.

Алгоритм BFS начинает обход с некоторой выбранной вершины. Он использует очередь для посещения вершин по уровням: сначала все соседи текущей вершины, затем их соседи, и так далее.

Визуально BFS можно представить как распространение волны от начальной вершины, слой за слоем.



Порядок выхода из функции: $\{5, 6, 7, 8, 9, 10, 2, 3, 4, 1\}$

Такой порядок обхода называется **уровневым (breadth-first) обходом дерева**

Реализация

```
void bfs(int start) {
  queue<int> q;
  q.push(start);
  visited[start] = true;

  while (!q.empty()) {
    int v = q.front();
    q.pop();

    for (int u : adj[v]) {
      if (!visited[u]) {
        visited[u] = true;
        q.push(u);
      }
    }
  }
}
```

Сложность

- **Временная сложность**: $O(V + E)$, где V — количество вершин, E — количество рёбер. Каждая вершина и каждое ребро посещается постоянное число раз.
- **Пространственная сложность**: $O(V)$ для хранения массива visited и очереди. В худшем случае (линейный граф) может потребоваться $O(V)$ очереди.

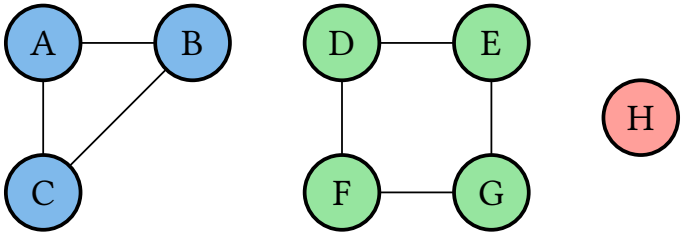
BFS работает оптимально по времени для невзвешенных графов, находя кратчайшие пути по количеству рёбер.

Практические задачи

- **Binary Tree Level Order Traversal**: LeetCode 102
- **Binary Tree Zigzag Level Order Traversal**: LeetCode 103
- **Walls and Gates**: LeetCode 286
- **Поиск в ширину**: acmp.ru 87

Компоненты связности

Дан неориентированный граф. Требуется найти все компоненты связности, а также определить к какой из компонент относится каждая вершина.



Компоненты связности графа сверху: $\{A, B, C\}$, $\{D, E, F, G\}$, $\{H\}$.

```
void bfs(int v, int u) {
    queue<int> q;
    q.push(v);
    visited[v] = u;

    while (!q.empty()) {
        int w = q.front();
        q.pop();
        for (int i : adj[w]) {
            if (!visited[i]) {
                visited[i] = u;
                q.push(i);
            }
        }
    }
}
```

```
vector<vector<int>> adj;
vector<int> visited;

int main(){
  // Считывание

  int n = adj.size();
  visited.assign(n, 0);
  for (int i = 0; i < n; ++i) {
    if (!visited[i]) {
      bfs(i, i);
    }
  }

  // Вывод
}
```

Работает за $O(V + E)$.

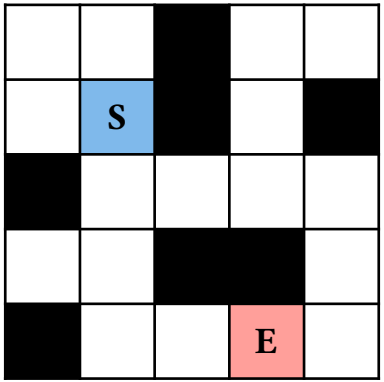
Практические задачи

- **Number of Islands**: LeetCode 200
- **Assigning Components**: SPOJ “PT07Y”
- **Граф**: acmp.ru 713
- **Телефонный справочник**: Timus 1002

Лабиринт

Вы находитесь в лабиринте, карту которого можно расположить на тетрадном листе. В клетки либо можно находиться самому, либо в ней находится стена. Ваша задача — найти кратчайший путь из лабиринта.

На примере снизу требуется найти кратчайший путь от клетки S, до клетки E.



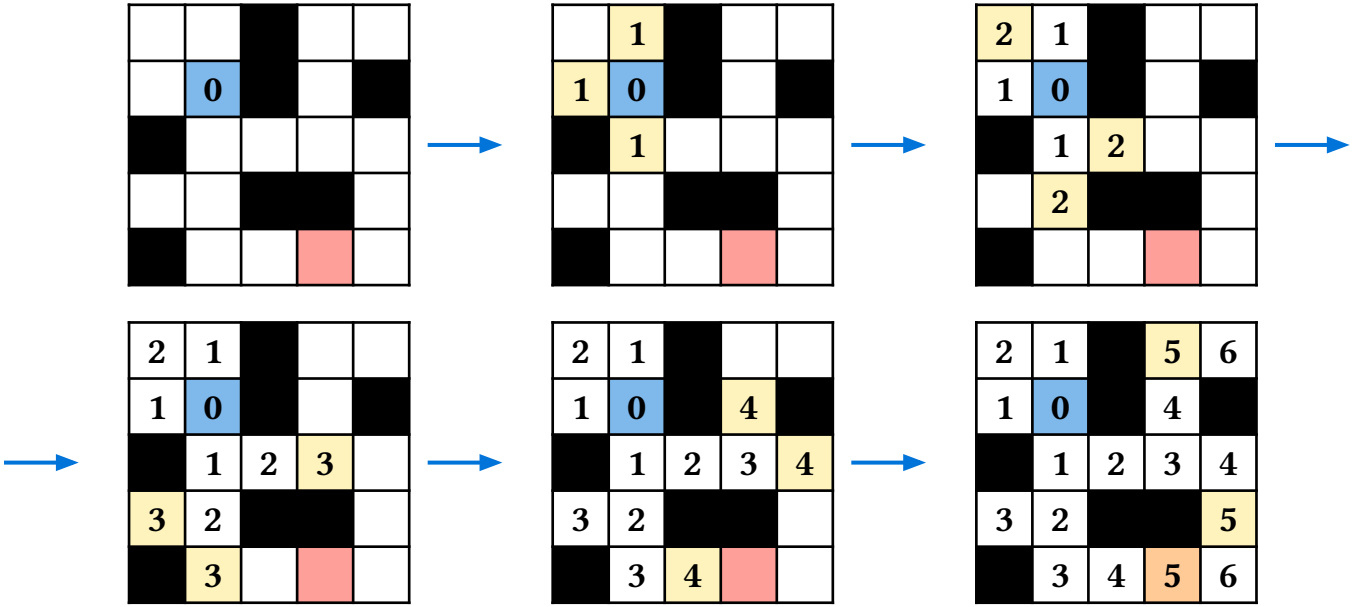
Решение

Алгоритм Ли (волновой алгоритм) - применение BFS для поиска кратчайшего пути в матрице, например в лабиринте.

Представим сетку матрицу граф, где каждая клетка - вершина, соседние клетки соединены рёбрами (обычно 4 направления: вверх, вниз, влево, вправо).

BFS “волна” распространяется от старта, каждая волна - новый уровень расстояния. В таблице ниже показан шаг-за-шага прогресс волнового алгоритма на данном лабиринте.

Запишем в стартовую клетку 0, а затем будем запускать бфс и записывать значение на 1 больше во все соседние клетки.


0
1
1 0
1
2 1
1 0
1 2
2
2 1
1 0
1 2 3
3 2
3
2 1
1 0 4
1 2 3 4
3 2
3 4
2 1 5 6
1 0 4
1 2 3 4
3 2 5
3 4 5 6

```
int dx[4] = {0,1,0,-1};
int dy[4] = {1,0,-1,0};
void bfs() {
    queue<pair<int,int>> q;
    q.push({sx, sy});
    dist[sx][sy] = 0;

    while (!q.empty()) {
        auto [x, y] = q.front();
        q.pop();
        if (x == ex && y == ey) return dist[x][y];

        for (int d = 0; d < 4; d++) {
            int nx = x + dx[d], ny = y + dy[d];
            if (nx >= 0 && nx < n && ny >= 0 && ny < m &&
                grid[nx][ny] == 0 && dist[nx][ny] == -1) {
                dist[nx][ny] = dist[x][y] + 1;
                q.push({nx, ny});
            }
        }
    }
}
```

Восстановление пути

После выполнения BFS матрица `dist` содержит расстояния от стартовой позиции. Для восстановления пути производим обратный обход: начиная с конечной позиции, ищем соседние клетки с расстоянием на 1 меньшим, пока не достигнем стартовой.

2	1		5	6
1	0		4	
	1	2	3	4
3	2			5
	3	4	5	6

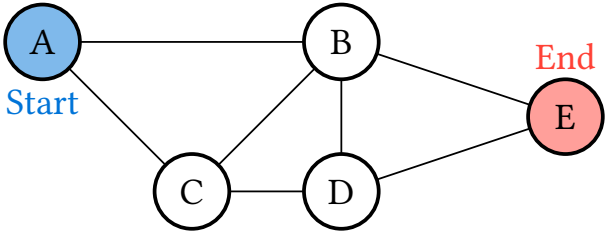
```
vector<pair<int,int>> path(int sx, int sy, int ex, int ey) {
    vector<pair<int,int>> path;
    int x = ex, y = ey;
    while (x != sx || y != sy) {
        path.push_back({x, y});
        for (int d = 0; d < 4; d++) {
            int nx = x + dx[d], ny = y + dy[d];
            if (nx >= 0 && nx < n && ny >= 0 && ny < m &&
                dist[nx][ny] == dist[x][y] - 1) {
                x = nx; y = ny;
                break;
            }
        }
    }
    path.push_back({sx, sy});
    reverse(path.begin(), path.end());
    return path;
}
```

Практические задачи

- **Лабиринт**: LeetCode 490
- **Кратчайший путь в двоичной матрице**: LeetCode 1091
- **Klorox**: SPOJ “KLORO”
- **Графы**: acmp.ru 712

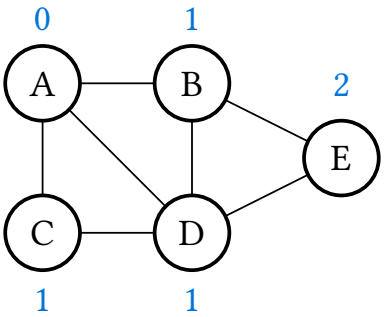
Кратчайший путь в невзвешенном графе

Дан невзвешенный граф. Требуется найти кратчайший путь (по количеству рёбер) между двумя вершинами start и end.



Решение

В стартовую вершину запишем 0. Во все последущие посещённые вершины будет записывать значение на 1 больше, чем в вершине из которой мы пришли.



Расстояния от вершины A — B:1, C:1, D:1, E:2.

```
vector<int> dist(n, -1);
dist[start] = 0;
queue<int> q;
q.push(start);

while (!q.empty()) {
    int v = q.front();
    q.pop();
    for (int u : adj[v]) {
        if (dist[u] == -1) {
            dist[u] = dist[v] + 1;
            q.push(u);
        }
    }
}
```

Работает за $O(V + E)$.

Практические задачи

- **Лестница слов**: LeetCode 127
- **Минимальная генетическая мутация**: LeetCode 433
- **Расстояние в графе**: acmp.ru 48
- **Открыть замок**: LeetCode 752
- **Обход графа**: acmp.ru 711